



Tribhuvan University
Faculty of Humanities and Social Sciences

“FOODLY (FOOD ORDERING APP)”
A PROJECT REPORT

Submitted to
Department of Bachelor in Computer Application
Ed-Mark College

In partial fulfilment of the requirements for the Bachelor's in
Computer Application

Submitted by
Himansi Shrestha (TU Reg No:6-2-727-14-2022)

Under the Supervision of
Mr. Shyam Maharjan

December, 2025



Tribhuvan University

Faculty of Humanities and Social Sciences

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **HIMANSI SHRESTHA** entitled “**FOODLY (FOOD ORDERING APP)**” in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

Mr. Shyam Maharjan

SUPERVISOR

Department of BCA

Ed-Mark College

Kalanki, Kathmandu



Tribhuvan University

Faculty of Humanities and Social Sciences

LETTER OF APPROVAL

This is to certify that this project prepared by **HIMANSI SHRESTHA** entitled “**FOODLY (FOOD ORDERING APP)**” in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

..... SIGNATURE Mr. Shyam Maharjan Supervisor Department of BCA Ed-Mark College Kalanki, Kathmandu	 SIGNATURE Er. Sanjay Kumar Sah Coordinator Department of BCA Ed-Mark College Kalanki, Kathmandu
..... SIGNATURE Internal Examiner	 SIGNATURE External Examiner

ABSTRACT

Foodly is a multi-vendor food ordering mobile application that enables users to order food from multiple restaurants through a single platform. The application allows customers to browse restaurants, explore menus, customize meals, place orders and make secure online payments. By providing a simple and intuitive interface, Foodly enhances user convenience and improves the overall food ordering experience.

The platform supports multiple user roles, including customers, restaurant vendors, delivery personnel, and a super admin. Restaurant vendors can manage menus, update prices, accept or reject orders, and monitor order status. Delivery personnel receive delivery requests, navigate pickup and drop locations, and update delivery progress. The super admin has centralized control over users, restaurants, orders, and system performance, ensuring smooth platform operation.

Foodly integrates secure payment processing to ensure reliability and efficiency. The application improves coordination between customers, restaurants, and delivery staff, reduces manual workload, and provides a scalable digital solution for modern food delivery services.

Keywords: *Online Food Delivery, Restaurant Management, Delivery Management System, Super Admin Panel, Secure Payments, Mobile Application*

ACKNOWLEDGEMENT

It is with great pleasure that we present this project report on "**Foodly (Food Ordering App)**". The successful completion of this work would not have been possible without the guidance, encouragement, and support of several individuals and institutions, to whom we express our heartfelt gratitude.

First and foremost, we extend our sincere appreciation to **Tribhuvan University** for providing us with the opportunity to undertake this project. We are also deeply grateful to **Ed-Mark College** and our esteemed **Principal Mr. Shiva Prasad Adhikari**, for fostering an environment that nurtures academic excellence and personal growth.

We owe a special debt of gratitude to our **Project Supervisor Mr. Shyam Maharjan** and **Program Coordinator Er. Sanjay Kumar Sah**, whose invaluable guidance, insightful advice, and unwavering support were instrumental in shaping this project. Their mentorship has been truly inspiring.

We also extend our appreciation to our teammates for their dedication, collaboration, and perseverance, which made this endeavor possible. A heartfelt thank you to our seniors for their valuable insights and to our friends for their constant encouragement and assistance throughout this journey.

Lastly, we are profoundly grateful to our families for their unwavering support, motivation, and belief in us. Their encouragement has been our greatest source of strength.

We hope this report meets the academic standards and serves as a meaningful contribution to our field of study.

Name of Students: Himansi Shrestha

TABLE OF CONTENT

ABSTRACT.....	i
ACKNOWLEDGEMENT	ii
LIST OF ABBREVIATIONS.....	v
LIST OF FIGURES	vi
LIST OF TABLES.....	vii
CHAPTER 1 INTRODUCTION	1
1.1 Introduction.....	1
1.2 Problem Statement.....	1
1.3 Objectives	2
1.4 Scope and Limitations	2
1.5 Development Methodology	3
1.6 Report Organization.....	4
CHAPTER 2 BACKGROUND STUDY AND LITERATURE REVIEW	5
2.1 Background Study.....	5
2.2 Literature Review	5
CHAPTER 3 SYSTEM ANALYSIS AND DESIGN	7
3.1 System Analysis.....	7
3.1.1 Requirement Analysis.....	7
i. Functional Requirement.....	7
ii. Non-Functional Requirement.....	8
3.1.2 Feasibility Analysis	9
3.1.3 Data Modeling: ER-Diagram.....	10
3.1.4 Process Modeling: DFD	11
3.2 System Design	13
3.2.1 Architectural design.....	13
3.2.2 Database Schema Design.....	14
3.2.3 Interface design.....	14
3.2.4 Physical DFD.....	19

3.3 Algorithm details	19
CHAPTER 4 IMPLEMENTATION.....	21
4.1 Implementation	21
4.1.1 Tools Used	21
4.1.2 Implementation details of module	22
4.2 Testing	23
4.2.1 Test Case for Unit Testing	23
4.2.2 Test case for System Testing	25
CHAPTER 5 CONCLUSION AND FUTURE RECOMMENEDATIONS	27
5.1. Lesson Learnt/Outcome	27
5.2. Conclusions.....	27
5.3. Future Recommendations	28
REFERENCES	

LIST OF ABBREVIATIONS

API	Application Programming Interface
Dart	Programming Language Used with Flutter
DB	Database
Flutter	Cross-Platform UI Software Development Kit
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model View Controller
MySQL	My Structured Query Language
Node.js	JavaScript Runtime Environment
SDK	Software Development Kit
SQL	Structured Query Language
UI	User Interface

LIST OF FIGURES

Figure 1: Development Methodology for the System.....	3
Figure 2 : Use case diagram for the system	8
Figure 3: ER-diagram of the System	10
Figure 4: Level 0 DFD.....	11
Figure 5: Level 1 DFD.....	12
Figure 6: Architectural design for System [6]	13
Figure 7: Database Schema Design of the system	14
Figure 8: Interface Design of Sign-up form.....	14
Figure 9: Interface Design of Login Form.....	15
Figure 10: Interface Design of Home page of user.....	15
Figure 11: Interface design for Admin Panel.....	16
Figure 12: Interface Design for Restaurant.....	16
Figure 13: Interface Design for Search Page	17
Figure 14: Interface Design for the Cart Page	17
Figure 15: Interface Design for Category	18
Figure 16:Interface Design for menu Items	18
Figure 17: Physical DFD of the system	19

LIST OF TABLES

Table 1: Unit testing for the User Sign-up.....	23
Table 2: Unit Testing for the User login.....	24
Table 3: Unit Testing for Restaurant Sign Up	24
Table 4: Unit testing for the Food Searching.....	24
Table 5: Unit testing for adding Food Product	24
Table 6:Unit Testing for order	24
Table 7: Unit testing for the sending order to the restaurant	25

CHAPTER 1

INTRODUCTION

1.1 Introduction

The multi-vendor online food ordering system allows customers to order food from multiple restaurants using a single application. Customers can browse restaurants, choose food items, place orders, and receive their meals at their doorstep. The system supports online payment as well as cash on delivery, making the ordering process easy and convenient for users.

With the growth of the internet and digital technology, many restaurants are shifting from traditional phone-based ordering to online platforms. This system helps restaurants reach more customers while reducing errors caused by manual order taking. Vendors can view orders in real time, manage food preparation, and update order status efficiently.

The system also reduces workload for restaurant staff by automating order management. All order details are stored in a database and displayed clearly to vendors and administrators. The major advantage of this multi-vendor food ordering app is its flexibility, as it supports multiple restaurants, smooth coordination, and faster service for customers.

1.2 Problem Statement

The rapid growth of online food delivery services has changed the way people order meals. However, customers, restaurants, and delivery personnel still face several challenges when using traditional or single-vendor food ordering systems:

- Food options from multiple restaurants are scattered across different platforms, making it hard to compare menus, prices, and offers.
- Many restaurants still take orders manually through phone calls, causing mistakes, delays, and confusion.
- Customers face limited choice and must spend extra time searching for restaurants and menu individually.

- Users find it difficult to discover restaurants or food items that match their preferences, budget, or location due to lack of recommendation systems.

1.3 Objectives

The system has following objectives:

- To develop a digital platform connecting multiple restaurants with customers.
- To improve overall efficiency, reduce errors, and enhance customer satisfaction compared to traditional food ordering methods.

1.4 Scope and Limitations

The system is a dynamic marketplace designed to connect customers, restaurants, and delivery personnel. The scope includes:

- **Detailed Vendor Information:** Provides comprehensive profiles for restaurants including menus, pricing, dish descriptions, dietary tags, and high-quality food images.
- **Secure User Management:** Enables customers, vendors, and delivery agents to create accounts and log in securely with role-based access control.
- **Intelligent Discovery:** Allows users to search for food items or restaurants and provides recommendations based on cuisine type and location.
- **Order & Cart Management:** Customers can add items from multiple categories, customize orders (e.g., extra toppings), and place orders in real time.
- **Vendor Dashboard:** Empowering restaurants to manage their digital storefronts, update item availability, process incoming orders, and view daily sales reports.
- **Admin Control Panel:** A centralized hub for the admin to approve new vendors, manage delivery personnel, oversee system-wide payments, and resolve disputes.

While the platform offers convenience, it is subject to the following limitations that users and stakeholders should be aware of:

- **Database Constraints:** The system can only suggest restaurants and dishes that are formally registered in its database, limiting the discovery of unregistered local eateries.
- **Algorithm Bias:** Recommendations are based on historical data and user preferences; if a user's order history is limited, the variety of suggestions may become narrow.

- **Operational Dependency:** The accuracy of "Estimated Time of Arrival" (ETA) is subject to external factors such as traffic, weather conditions, and restaurant preparation delays which may not update instantly.
- **Quality Control:** The system facilitates the transaction and delivery but cannot physically guarantee the hygiene, taste, or portion accuracy of the food prepared by the vendors.
- **Technical Dependencies:** The platform relies on stable internet connectivity for all four parties; any downtime in third-party services (Maps or Payment Gateways) will result in feature failure.
- **Geographical Boundaries:** Service availability is strictly limited to defined delivery areas users outside these zones will be unable to place orders.

1.5 Development Methodology

The system is being built using the Agile method, a flexible and practical approach that allows the project to evolve based on real user needs. This method was chosen because a multi-vendor app requires constant adjustments to satisfy customers, restaurants, and delivery drivers alike. Instead of building the entire platform at once, the work is divided into small, manageable stages called sprints.

This method emphasis on continuous testing and feedback. By reviewing progress at the end of every sprint, we can catch technical issues early and fix them before they grow. Ultimately, the Agile method ensures that the app doesn't just function well, but remains a simple, reliable, and user-friendly tool that truly fits the fast-paced nature of a food ordering marketplace.

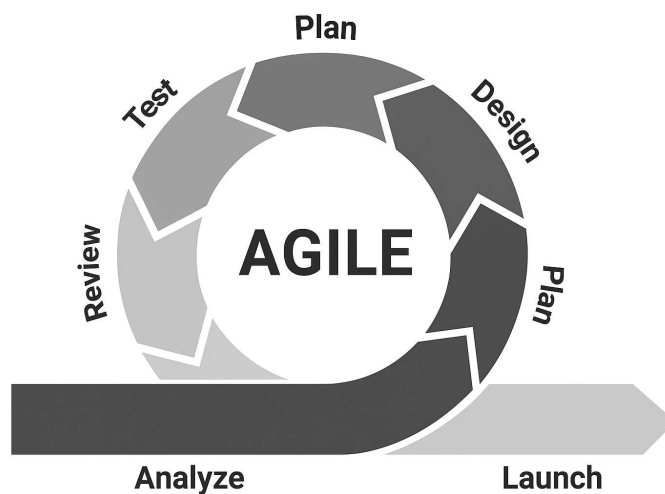


Figure 1: Development Methodology for the System [7]

1.6 Report Organization

The first part of the report gives an overview of the entire project, explaining its scope, limitations, problem statement, and objectives.

The second chapter covers a background study, explaining basic theories, general ideas, and key terms related to the project. It also includes a review of similar projects and research done by other people.

In the third chapter, the focus is on system analysis and design. It describes the project's functional and non-functional requirements using use case and system diagrams. A feasibility study is also done to check if the project is affordable and if the developers have the right skills to complete it. The technical, operational, and economic aspects are considered. This chapter also includes the system's design, such as data modeling with ER diagrams and process modeling using Data Flow Diagrams. The architectural design, database design, and user interface design are explained as well.

The fourth chapter talks about the implementation and testing of the Book Recommendation System. It covers the use of tools like CASE tools, programming languages, and database platforms.

Lastly, the fifth chapter provides the conclusion and future recommendations. It sums up the project's results and gives the developers' views on the work. It also suggests possible improvements or developments for the future.

CHAPTER 2

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

With the growth of mobile technology, customers increasingly depend on online platforms to discover restaurants, browse diverse menus, and make secure payments. Traditional methods of ordering such as manual over-the-counter requests, phone calls, or physically visiting multiple restaurants to compare prices are time-consuming and often lead to errors in order processing or a lack of clear information regarding food availability. This creates a critical need for a reliable, multi-vendor system that provides quick access to a wide variety of culinary options in one centralized location.

Mobile applications built using Flutter offer a high-performance and visually engaging interface, allowing customers to seamlessly navigate between different restaurant storefronts, view high-quality food images, and manage their orders across both Android and iOS devices from a single codebase. Meanwhile, backend technologies like Node.js provide an efficient, event-driven environment for handling real-time requests. Paired with MySQL, a robust relational database, the system ensures secure data storage, accurate transaction records, and efficient management of complex information like vendor inventories, customer profiles, and delivery logs.

As the food market becomes more competitive, local restaurants and vendors require modern digital tools to showcase their menus, reach a broader customer base, and manage high volumes of orders without manual overhead. A food ordering app facilitates this digital transformation by offering a responsive and easy-to-use platform for customers, vendors, and administrators alike. This background highlights the importance of building an integrated ordering system that improves accessibility for users, enhances operational efficiency for businesses, and supports the overall growth of the digital economy.

2.2 Literature Review

The growth of e-commerce and mobile technologies has significantly changed the food service industry, enabling the rise of online food ordering systems. Researchers have consistently emphasized that digital platforms improve efficiency, reduce human errors,

and enhance customer satisfaction by replacing traditional manual ordering methods with automated solutions. Patel and Desai, argue that automation in food ordering ensures accuracy and faster processing, which is essential in busy urban lifestyles. [1]

Another key area of research is usability and adoption of mobile applications in the food ordering domain. Sharma, found that mobile-based systems with simple navigation and intuitive design improve customer experience, leading to higher adoption rates. Their study revealed that user-friendly features such as menu search, order customization, and instant notifications are crucial in retaining customers.[2]

Case studies of existing platforms like UberEats, Zomato, and Foodpanda also provide useful insights into business models and operational challenges. Research by Kumar and Roy indicates that while commission-based systems allow large-scale operations, they often impose financial burdens on small and medium-sized restaurants, limiting their profitability. Similarly, studies have shown that local vendors in developing countries struggle to access such platforms due to high costs and limited digital.[3]

Meanwhile, Wang, highlighted the importance of secure payment gateways and encrypted data transmission in online food ordering systems. Security has become a major concern as customers increasingly use digital wallets and online banking; therefore, implementing strong encryption and multi-factor authentication is essential for building customer confidence.[4]

Highlighted the importance of multi-vendor platforms, stating that they provide a centralized marketplace where customers can access a variety of restaurants and cuisines, thereby increasing customer convenience and vendor visibility. This aligns with the findings of Ali and Khan, who demonstrated that integrating real-time order tracking and notifications not only improves service transparency but also strengthens customer trust and loyalty.[5]

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

The purpose of the System Analysis is to establish a comprehensive food ordering platform that facilitates seamless interactions between customers, restaurant vendors, and delivery personnel.

The platform features a dynamic storefront where customers can browse multiple restaurants and view real-time pricing and availability. To ensure smooth operations, the system is governed by three primary administrative roles:

- **System Administrator:** Holds the highest level of access to manage the entire system, including user accounts and restaurant onboarding.
- **Restaurant Manager:** Responsible for managing their specific digital menu, updating product prices, and processing incoming orders.
- **Delivery Manager:** Oversees the assignment of delivery tasks and monitors order fulfilment status to ensure timely service.

3.1.1 Requirement Analysis

i. Functional Requirement

Functional Requirement of the system are:

1. **User Authentication:** The app must provide secure login and registration for customers, vendors, delivery personnel, and admin with role-based dashboards.
2. **Customer Features:** Customers should be able to browse or search for products, view detailed descriptions, add items to cart, place orders, make payments, and track order status in real time.
3. **Vendor Features:** Vendors can register their business, create and manage their profiles, add/edit/delete products, view new orders, update order status, and analyze sales reports.
4. **Delivery Features:** Delivery personnel should be able to accept assigned deliveries, use maps for navigation, and update delivery status (e.g., picked up, delivered).
5. **Admin Features:** The admin must oversee the entire system, approve or reject vendors and products, manage all user roles, monitor transactions, and view analytics dashboards.

- 6. Payment Integration:** The system must support multiple payment gateways like Stripe and others to ensure smooth financial transactions.
- 7. Search & Filter:** Customers must be able to search for products and filter them by category, price, or rating to improve product discovery.

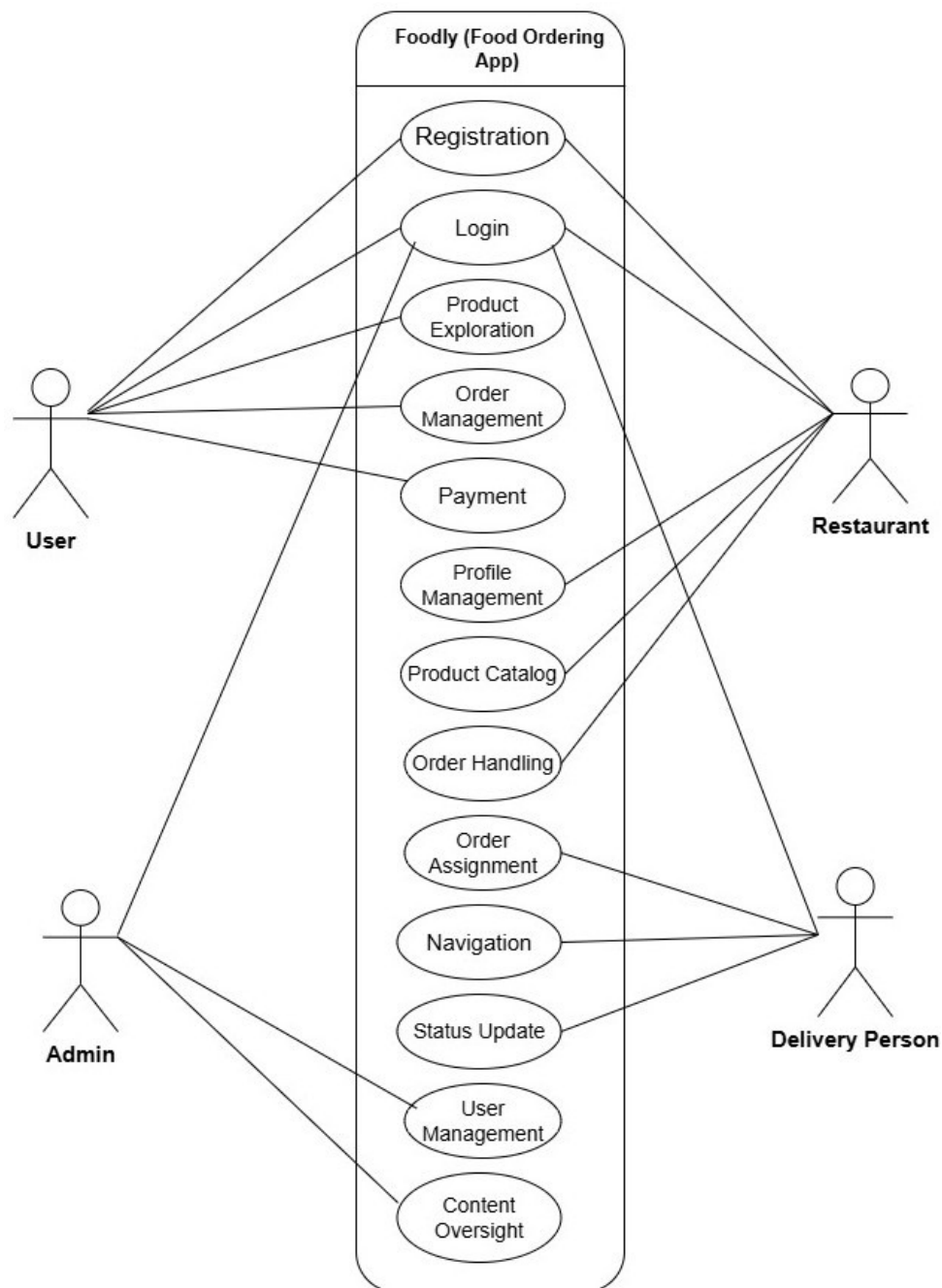


Figure 2 : Use case diagram for the system

ii. Non-Functional Requirement

Some common non-functional requirements of this system are:

1. Usability: The interface must be simple and intuitive so customers, vendors, and delivery personnel can easily use it without special training.

2. Reliability: The system must ensure accurate order processing and maintain regular backups to prevent data loss.

3. Scalability: The app must scale smoothly as the number of users, vendors, and orders increases without affecting performance.

4. Maintainability: The system must use modular and clean code so developers can easily add features, fix bugs, or perform updates.

3.1.2 Feasibility Analysis

A feasibility study is a way to evaluate whether or not a project plan could be successful. A feasibility study evaluates the practicality of your project plan in order to judge whether or not you're able to move forward with the project. This study is crucial for decision-makers to assess whether the project should proceed, be modified, or be abandoned. The feasibility of the School Management System project in terms of technical, operational, economic, social, legal, and environmental aspects.

1. Technical Feasibility

The project is technically feasible because it uses modern cross-platform technologies such as Flutter and Firebase/Node.js backend. Flutter allows the app to run on Android, iOS, and Web with a single codebase, reducing development time and effort. APIs make it possible to handle real-time updates (orders, tracking, notifications) and integrate multiple payment gateways (Stripe etc.). Since smartphones and internet penetration are widely available, both vendors and customers can easily access the system. The use of scalable architecture ensures that the app can handle an increasing number of users, vendors, and transactions without major performance issues.

2. Economic Feasibility

By automating vendor approvals, order processing, and payment management, the system reduces the need for large administrative staff. Vendors benefit from digital sales channels without needing to invest in their own e-commerce platforms. Customers save time and cost by shopping online, while the platform earns revenue from commissions,

advertisements, or subscription models. Overall, the system provides a cost-effective and sustainable solution compared to manual or traditional e-commerce setups.

3. Operational Feasibility

The system is practical and easy to operate for all user groups. Customers can use a simple interface to browse products, place orders, and track deliveries. Vendors can manage their stores and orders with minimal training. Delivery personnel can update order statuses using a dedicated app with map integration. The admin panel allows system administrators to monitor vendors, approve products, and oversee transactions. Since most people are already familiar with online shopping apps, the adoption of the system requires little user training, making it highly operable in real-world scenarios.

3.1.3 Data Modeling: ER-Diagram

ER-diagram consists of collections of basic objects called entities and relationship among these objects. ER-diagram of the system is:

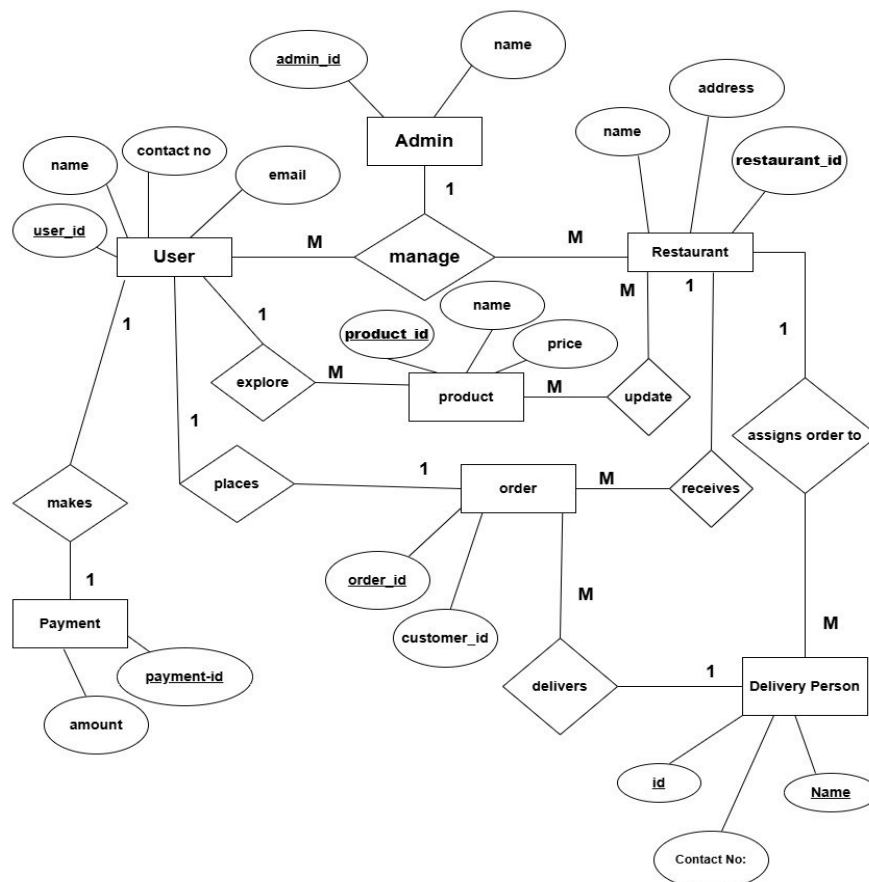


Figure 3: ER-diagram of the System

3.1.4 Process Modeling: DFD

A Data Flow Diagram (DFD) can be referred to as a Process Model. Initially, a context diagram is drawn, which provides a simple representation of the entire system. This is followed by the level 1 diagram.

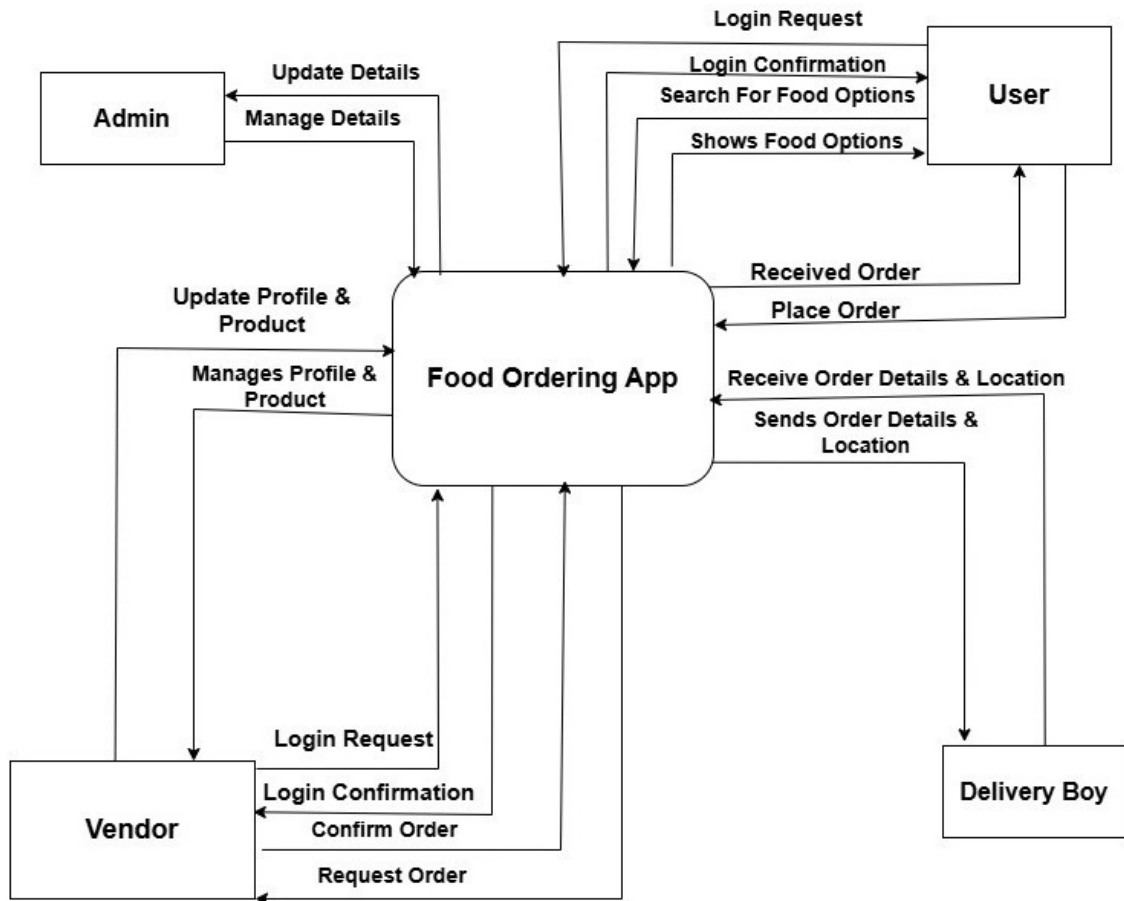


Figure 4: Level 0 DFD

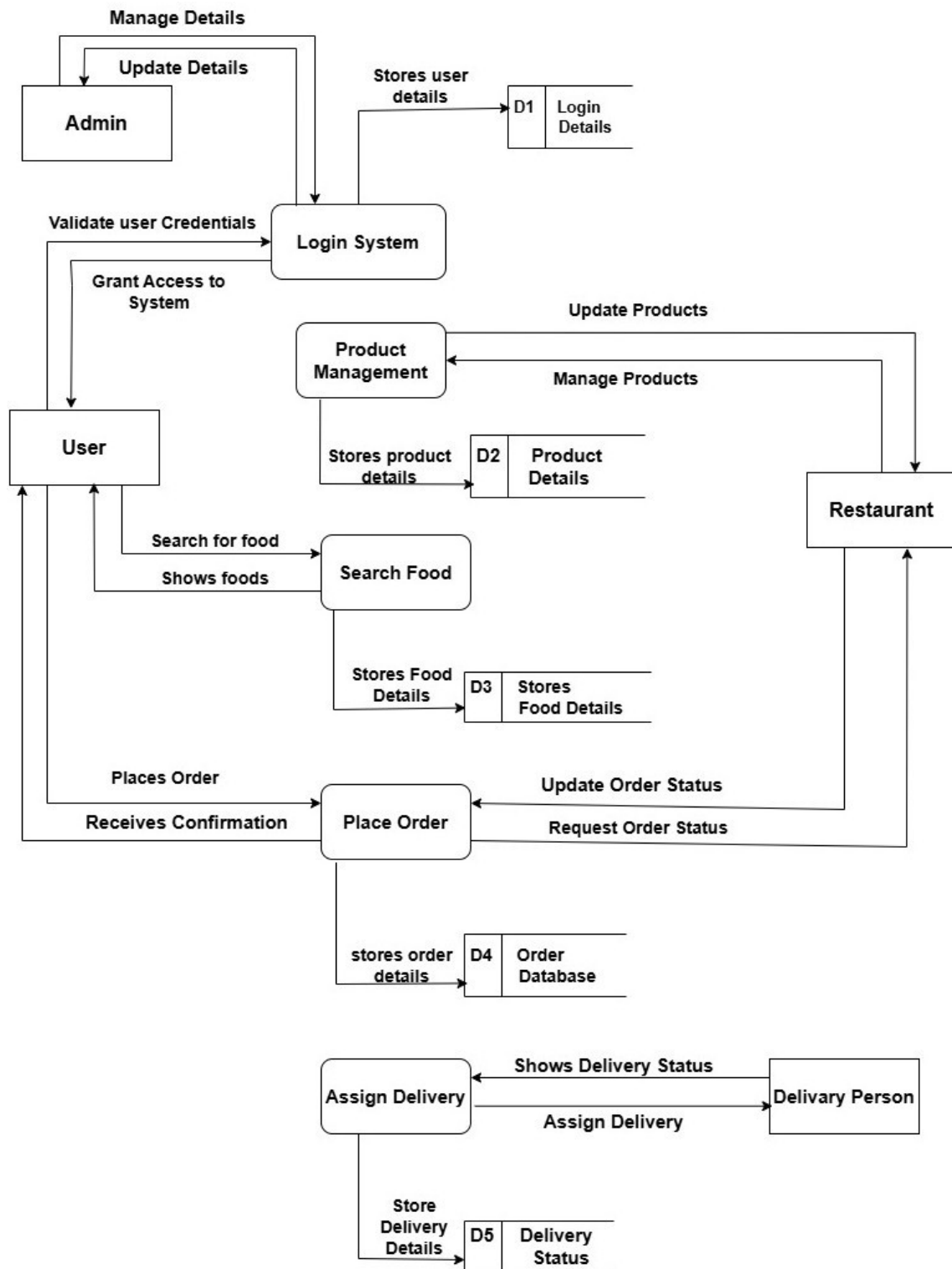


Figure 5: Level 1 DFD

3.2 System Design

3.2.1 Architectural design

Architecture design is the process of figuring out the different parts that make up a system and how these parts will work together. It involves determining how these parts will communicate and be controlled to create a well-organized and efficient system. Architectural design ensures that all the pieces fit together seamlessly to create a reliable and effective system.

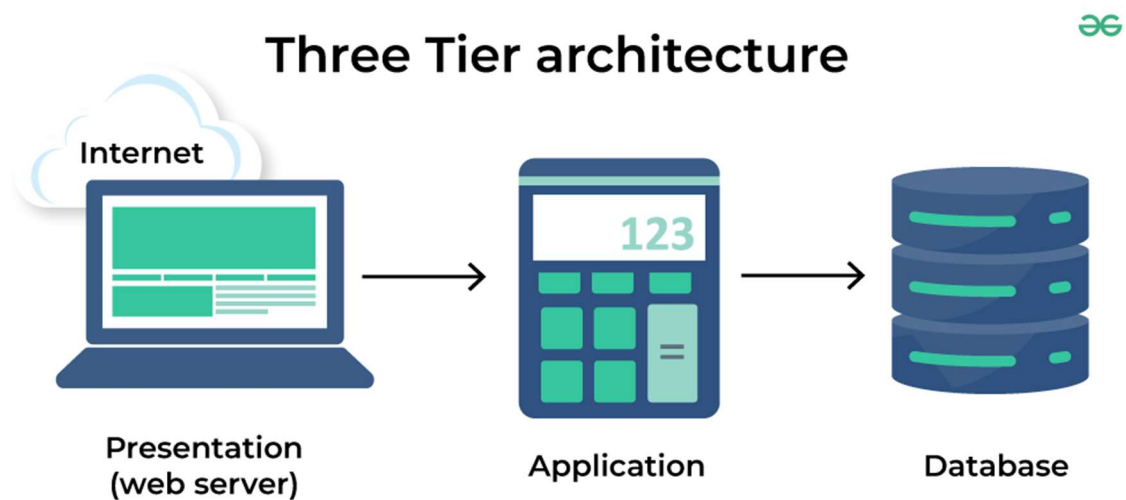


Figure 6: Architectural design for System [6]

The system is built using a Three-Tier Architecture, a highly organized design that separates the application into three distinct logical layers: the presentation tier, the application tier, and the data tier. The presentation tier is developed using Flutter, providing a cross-platform mobile application that allows customers to browse menus and vendors to manage orders without interacting directly with the database. The middle application tier, powered by Node.js, acts as the brain of the system and handles the business logic, such as validating user sessions, calculating totals, and managing real-time notifications, while serving as a secure gateway for all communication. Finally, the data tier utilizes MySQL to store and manage critical information, including user profiles, restaurant inventories, and transaction history, in a structured relational format.

3.2.2 Database Schema Design

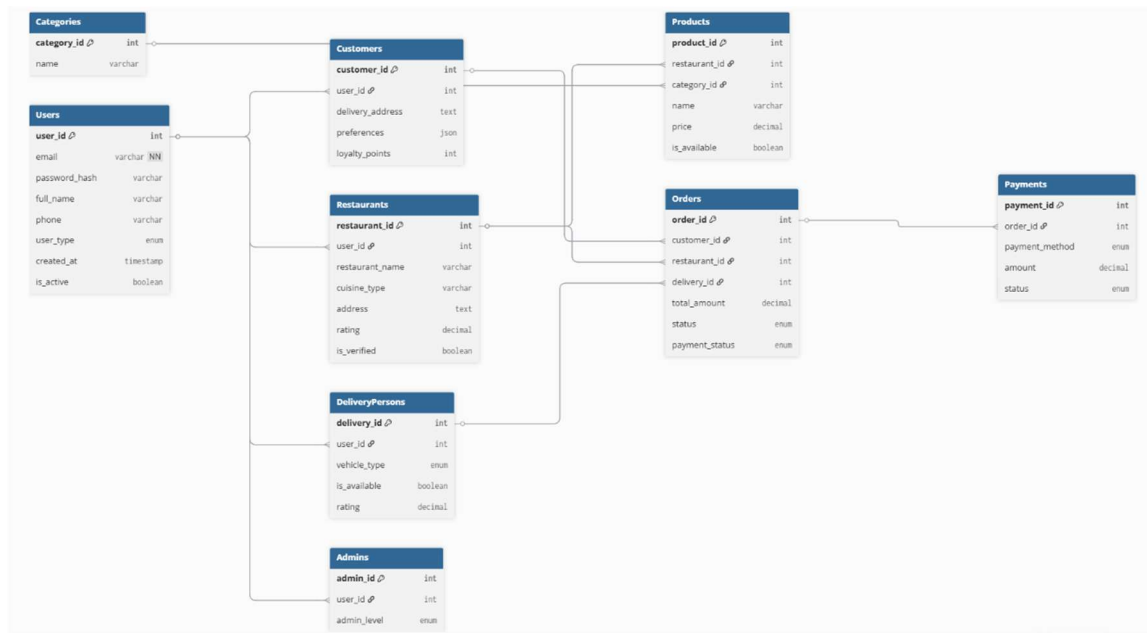


Figure 7: Database Schema Design of the system

3.2.3 Interface design

The interface design for the sign-up form is presented within a rounded rectangular container. It includes the following elements:

- Header:** "SignUp here" in bold black text.
- Input Fields:** Five rounded rectangular boxes for user input, each with a label in bold black text:
 - Full Name
 - Email
 - Contact No:
 - Pasword
 - Confirm Password
- Action Button:** A "Sign Up" button located at the bottom of the form.

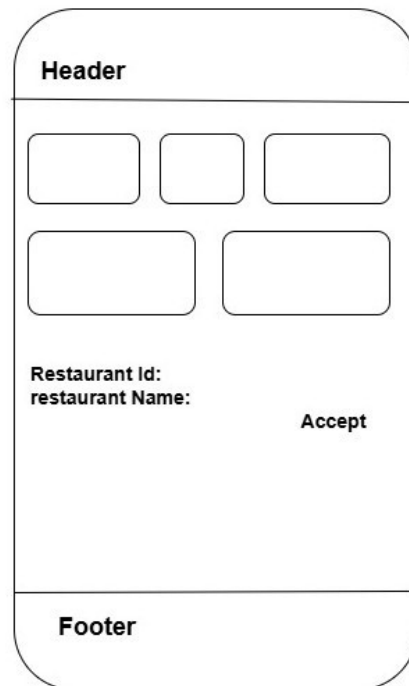
Figure 8: Interface Design of Sign-up form

A login form interface design within a rounded rectangle. At the top, the text "Login here" is centered. Below it, there are two input fields: the first is labeled "Contact No:" and the second is labeled "Pasword". At the bottom of the form is a "Login" button.

Figure 9: Interface Design of Login Form

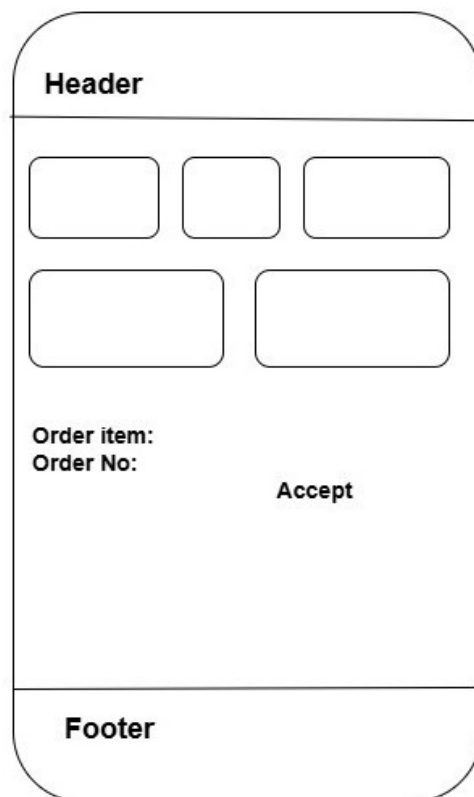
A home page interface design within a rounded rectangle. It is divided into sections: a "Header" section at the top; a section with three small square icons; a "Recommended Food" section with two rounded rectangular boxes; a "NearBy Food" section with two rounded rectangular boxes; and a "Footer" section at the bottom.

Figure 10: Interface Design of Home page of user



The interface for the Admin Panel is a vertical rectangle with rounded corners, divided into three main sections by horizontal lines. The top section is labeled "Header". Below the header, there are five rectangular input fields arranged in two rows: three in the top row and two in the bottom row. Below these fields, the text "Restaurant Id:" and "restaurant Name:" is displayed on the left, and an "Accept" button is on the right. The bottom section is labeled "Footer".

Figure 11: Interface design for Admin Panel



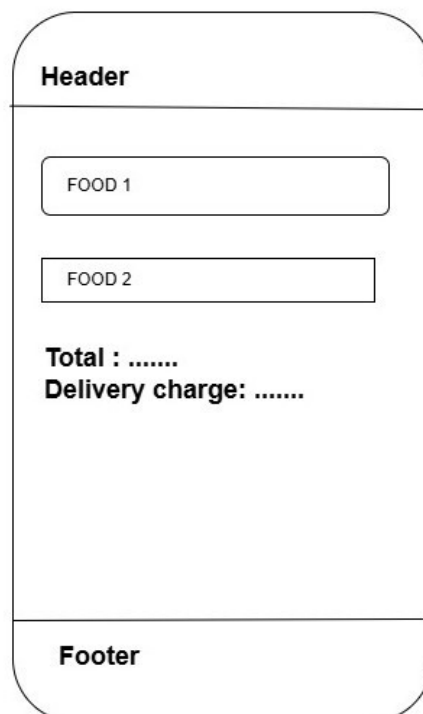
The interface for the Restaurant is a vertical rectangle with rounded corners, divided into three main sections by horizontal lines. The top section is labeled "Header". Below the header, there are five rectangular input fields arranged in two rows: three in the top row and two in the bottom row. Below these fields, the text "Order item:" and "Order No:" is displayed on the left, and an "Accept" button is on the right. The bottom section is labeled "Footer".

Figure 12: Interface Design for Restaurant



The diagram shows a mobile application interface for a search page. It is a vertical rectangle with rounded corners, divided into three horizontal sections. The top section is labeled "Header" and contains a search bar with the text "Search". Below the search bar are three empty rectangular boxes, likely for displaying search results. The bottom section is labeled "Footer".

Figure 13: Interface Design for Search Page



The diagram shows a mobile application interface for a cart page. It is a vertical rectangle with rounded corners, divided into three horizontal sections. The top section is labeled "Header". The middle section contains two items: "FOOD 1" and "FOOD 2", each in a rectangular box. Below these items, the text "Total :" and "Delivery charge:" is displayed. The bottom section is labeled "Footer".

Figure 14: Interface Design for the Cart Page

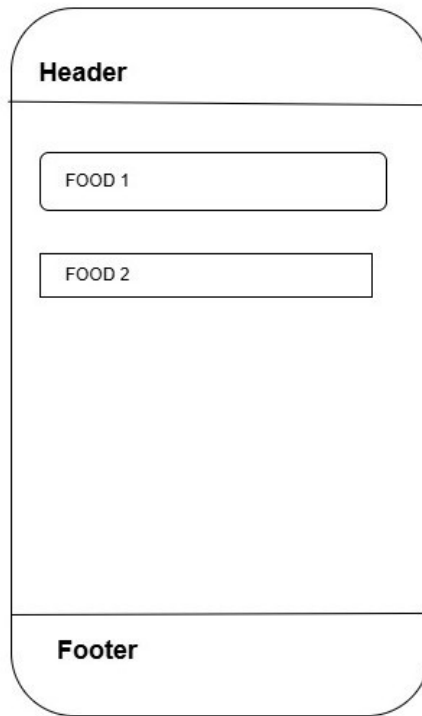


Figure 15: Interface Design for Category

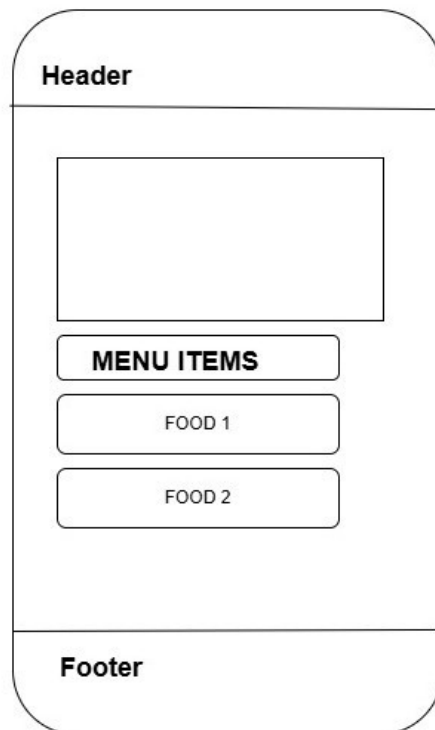


Figure 16:Interface Design for menu Items

3.2.4 Physical DFD

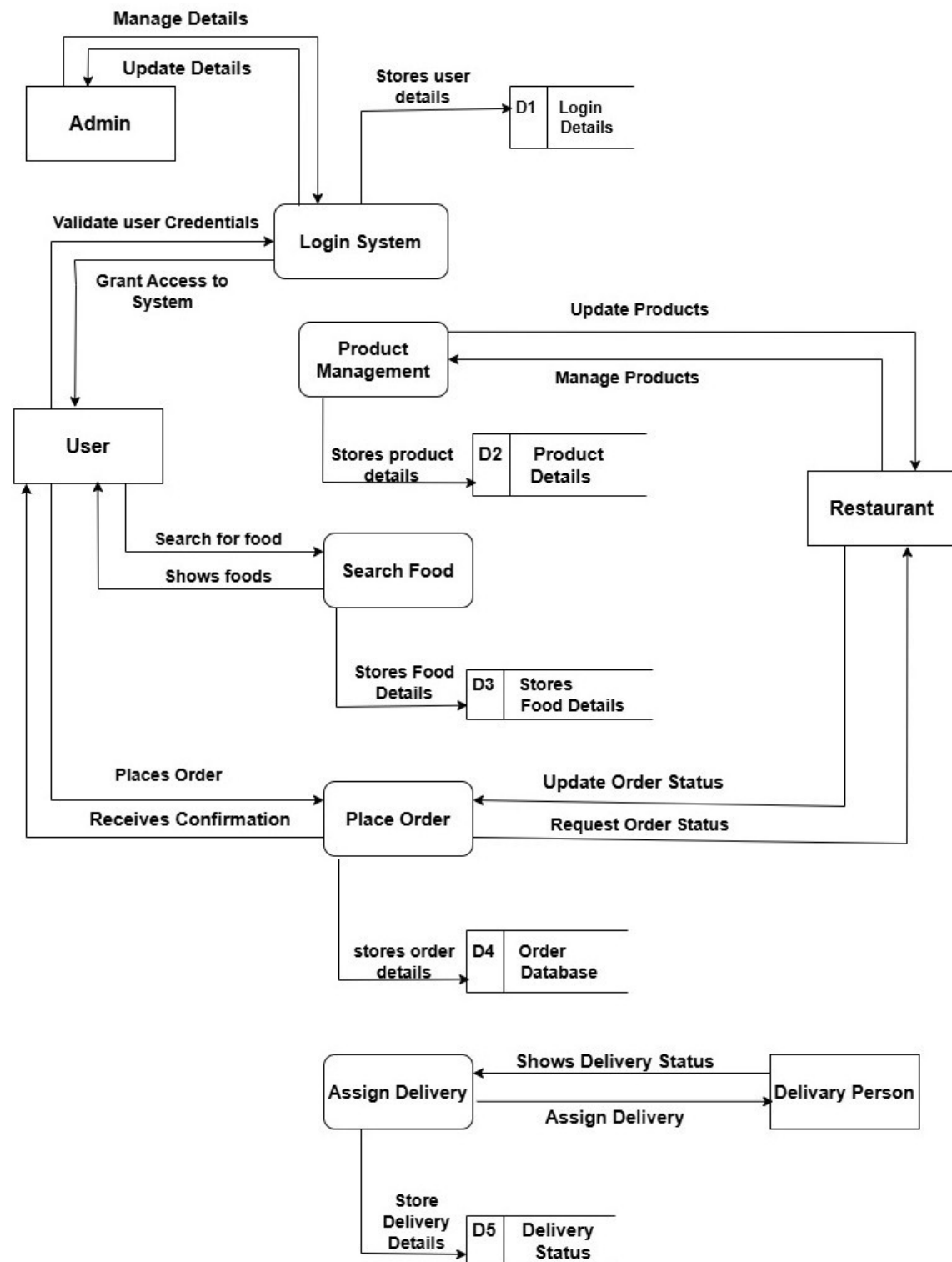


Figure 17: Physical DFD of the system

3.3 Algorithm details

- Fuzzy Search (Levenshtein Distance)

Fuzzy search is a technique used to find matches that are approximately equal rather than exactly equal.

One of the most common algorithms behind fuzzy search is Levenshtein Distance.

Levenshtein Distance measures the minimum number of single-character edits required to change one string into another.

Allowed edits:

1. Insertion (add a character)
2. Deletion (remove a character)
3. Substitution (replace a character)

Why Fuzzy Search is Useful

Fuzzy search helps when:

- Users make typos
- Spellings vary
- Partial or approximate matches are acceptable

Common use cases

- Search bars
- Product name matching
- Autocomplete
- Customer name lookup
- Address matching

How Levenshtein Distance Works (Concept)

The algorithm uses dynamic programming to compare two strings character by character and build a cost matrix.

- Lower distance= more similar strings
- Distance = 0 (exact match)

CHAPTER 4

IMPLEMENTATION

4.1 Implementation

In this phase, the theoretical design is transformed into a working system. This is a crucial stage in creating a successful system and building user confidence in its functionality. The actual implementation of the system occurs during this phase. Once it is confirmed that the system meets the desired specifications, it is ready for real-world use. This phase ensures that the new system functions as intended and meets the needs of its users.

4.1.1 Tools Used

Tools that are used while designing this website are:

Front end: Flutter, Dart

Backend: Node.js, Express

Database: MySQL

Flutter

In this system, Flutter is used as the primary framework to build a high-performance, cross-platform application for Android, iOS, and the Web from a single codebase. Flutter provides a rich set of pre-designed widgets that allow for the creation of a beautiful and responsive user interface. It acts as the structure and the skin of the application, ensuring that the content is easy to navigate, visually consistent across all devices, and simple to update.

Dart

Flutter uses the Dart programming language, which is optimized for fast apps on any platform. In the system, Dart handles the client-side logic, managing how users interact with food menus, cart selections, and real-time tracking screens. It ensures smooth animations and a "native" feel that makes the app highly responsive to user input.

Node.js

Node.js is a server-side runtime environment used for building fast and scalable network applications. In this system, Node.js is used to create a dynamic backend. It is a powerful tool that runs on the web server to process requests from the Flutter app. Because it is non-blocking and event-driven, it is perfectly suited for handling multiple orders and real-time notifications simultaneously.

Express

Express is a lightweight framework for Node.js used to organize the web application into an MVC architecture. It is implemented to handle routing (API endpoints), manage user sessions, and facilitate communication between the Flutter frontend and the MySQL database.

MySQL

Every robust marketplace requires a structured database for storing critical information. MySQL, a widely popular relational database management system, is used here. In this project, MySQL is utilized to neatly organize data into tables—such as Users, Restaurants, Orders, and Payments—and create clear relationships between them using foreign keys. This ensures data integrity and allows for fast searching and sorting, enabling the system to process transactions accurately and retrieve information quickly when needed.

4.1.2 Implementation details of module

According to the system requirements, the multi-vendor application is divided into four main functional modules:

- **Admin Module**
- **User Module**
- **Vendor Module**
- **Delivery Module**

These modules work together to ensure efficient platform management, smooth business operations, and reliable order fulfillment.

Admin Module

It provides centralized control over the entire system. It allows administrators to securely log in and manage users, vendors, delivery personnel, products, orders, commissions, and payments. Administrators can approve vendor registrations, monitor order activities, manage promotional content, and ensure system security and data integrity.

User Module

It offers a user-friendly interface for customers to browse products or services from multiple vendors. Users can register, log in securely, search and filter items, place orders,

make online payments, track order status, and manage their profiles through a personal dashboard.

Vendor Module

It enables sellers to register their businesses and manage their offerings. Vendors can add, update, and remove products, manage pricing and availability, process incoming orders, update order statuses, and view sales reports through a dedicated vendor dashboard.

Delivery Module

It supports delivery personnel in handling order fulfillment. Delivery agents can receive assigned orders, view pickup and delivery details, update delivery status in real time, and confirm successful order completion, ensuring timely and efficient deliveries.

4.2 Testing

Testing is a process of executing a program with the intent of finding an error. A good test case is one that has high probability of finding an undiscovered error. The primary objective for test case design is to derive a set of tests that has the highest likelihood for uncovering defects in software.

4.2.1 Test Case for Unit Testing

Unit Testing was performed to validate the correctness of individual functions and methods within each module

Table 1: Unit testing for the User Sign-up

Test case Id	Test Scenario	Test Steps	Test Data	Expected Result	Pass/Fail
T1	Sign-up	Sign-up with the valid credentials	Name., Email, password, confirm passwords	Registered Successfully	Passed
T2	Sign-up	Sign-up with invalid credentials	Name, Email= Null, password	Email is empty	Failed

Table 2: Unit Testing for the User login

Test case Id	Test Scenario	Test Steps	Test Data	Expected Result	Pass/Fail
T3	Login	Login with the invalid credentials	Contact No=8 no, Password	Invalid number displayed	Failed
T4	Login	Login with the valid credentials	Contact No., Password	Login Successfully	Passed

Table 3: Unit Testing for Restaurant Sign Up

Test case Id	Test Scenario	Test Steps	Test Data	Expected Result	Pass/Fail
T5	Signup	Login with the valid credentials	Email, Password, contact No	Wait for admin approval	Passed

Table 4: Unit testing for the Food Searching

Test case id	Test scenario	Test steps	Test data	Expected Result	Pass/fail
T6	Food searching	Go to search	Name of the food	Result not shown	Failed
T7	Food Searching	Go to search	Name of the food	Related food shown	Passed

Table 5: Unit testing for adding Food Product

Test case id	Test scenario	Test steps	Test data	Expected Result	Pass/Fail
T8	Add Product	Go to the Product and click in add details	Fill the every items in the details	Packages added successfully	Passed

Table 6:Unit Testing for order

Test case id	Test scenario	Test steps	Test data	Expected Result	Pass/Fail
T9	Add food to the cart	Go to food menu and	Data didn't insert to the cart	Error to add in cart	Failed

		click add to the cart			
T10	Add food to the cart	Go to food menu and click add to the cart	Data insert to the cart	Successfully Inserted	Passed

Table 7: Unit testing for the sending order to the restaurant

Test case id	Test scenario	Test steps	Test data	Expected Result	Pass/Fail
T11	Added food to the cart	Go to your cart and click confirm order	Make payment	Confirmed order	Passed

4.2.2 Test case for System Testing

System testing tests the integration of each module in the system. We have tested whether the entire system is working properly or not. Different verifications and validations are done by giving input values to the system and by comparing with expected output.

There are many modules involved in the system testing as whole. In the system testing overall the system is tested and analyse whether the system is running and performing smoothly or not, below table show the system testing for this webapp.

Table 8: Test for the system testing

Test case id	Test cases	Test input	Expected outcome	Outcomes	Test status
T1	Sign-up	Sign-up with the valid credentials	Name., Email, password, confirm passwords	Registered Successfully	Passed
T4	Login	Login with the valid credentials	Contact No., Password	Login Successfully	Passed

T5	Signup	Login with the valid credentials	Email, Password, contact No	Wait for admin approval	Passed
T7	Food Searching	Go to search	Name of the food	Related food shown	Passed
T8	Add Product	Go to the Product and click in add details	Fill the every items in the details	Packages added successfully	Passed
T10	Add food to the cart	Go to food menu and click add to the cart	Data insert to the cart	Successfully Inserted	Passed
T11	Added food to the cart	Go to your cart and click confirm order	Make payment	Confirmed order	Passed

CHAPTER 5

CONCLUSION AND FUTURE RECOMMENEDATIONS

5.1. Lesson Learnt/Outcome

- **Cross-Platform Performance:** Successfully used Flutter to deliver a high-performance, native-like experience on both Android and iOS from a single codebase, significantly reducing development overhead.
- **Reliable Data Transactions:** Mastered MySQL relational logic to ensure that complex multi-vendor orders and financial payments are processed with 100% accuracy and zero data loss.
- **Secure Multi-Role Access:** Implemented a robust security layer using JWT and middleware to ensure that Customers, Vendors, and Admins can only access their own specific data and dashboards.
- **Modular System Design:** Achieved a clean Three-Tier Architecture that separates the user interface from the database, making it easy to update or scale individual parts of the app without breaking the whole system.
- **Automated Business Workflow:** Transformed manual food ordering into an automated digital ecosystem that manages everything from menu updates to driver assignments and commission calculations.
- **Production-Ready Solution:** Delivered a fully functional, professional-grade marketplace that proves the effectiveness of the Flutter-Node-MySQL stack for modern e-commerce.

5.2. Conclusions

In conclusion, this project has successfully delivered a robust and efficient multi-vendor food ordering platform that bridges the gap between local restaurants, delivery partners, and customers. The primary objective was to create a seamless system where users can discover a wide variety of cuisines, while vendors gain a digital storefront to manage their businesses and delivery agents receive optimized task assignments. By implementing a Three-Tier Architecture using Flutter, Node.js, and MySQL, the system provides a high-performance, cross-platform experience that ensures data integrity and real-time order tracking.

Furthermore, the platform empowers administrators with a centralized control centre to manage the entire marketplace, including the ability to onboard vendors and monitor financial transactions. These goals have been fully realized, resulting in a scalable solution that provides tailored food recommendations to users and efficient management tools for stakeholders. This system not only streamlines the food ordering process but also establishes a reliable digital foundation that can adapt to future market demands and technological advancements.

5.3. Future Recommendations

For the long-term sustenance and scalability of the multi-vendor platform, the incorporation of AI-based personalization would enable the platform to understand user preferences as well as their ordering trends in order to suggest food based on those factors, thus increasing platform adoption. Improving the communication component by adding support for chat services via WebSockets would enable direct communication between customers and food delivery personnel. Also, the concept of subscription for free delivery would be introduced.

On a technical and functional basis, it is possible to improve the system further by implementing smart route algorithms that could assist delivery partners in their routes, thus cutting down delivery time and fuel expenses. The vendors could also be empowered to make smart decisions regarding their business, as advanced data analytics could be provided to them based on sale trends and customer feedback. On a functional basis, the user interface could also be revamped to include voice search capabilities and accessibility options, thus allowing all users to access the application effectively.

REFERENCES

- [1] S. Patel and A. Desai, "Automation in Online Food Ordering Systems: A Review," *International Journal of Computer Applications*, vol. 183, no. 15, pp. 25–29, 2021.
- [2] A. Sharma, V. Kaur, and S. Mehta, "Mobile-Based Food Ordering Systems: Usability and Adoption Factors," *Proc. Int. Conf. Smart Computing and Communications (ICSCC)*, pp. 135–140, 2020.
- [3] S. Kumar and A. Roy, "Business Models in Online Food Delivery: A Case Study of UberEats and Zomato," *International Journal of Business Research*, vol. 19, no. 2, pp. 75–83, 2020.
- [4] Y. Wang, L. Zhang, and H. Li, "Secure Payment Gateway Design for E-Commerce Applications," *IEEE Transactions on Consumer Electronics*, vol. 65, no. 4, pp. 455–462, 2019.
- [5] H. Ali and M. Khan, "Enhancing Food Delivery with Real-Time Tracking," *IEEE Access*, vol. 9, pp. 113456–113464, 2021.
- [6] Three-Tier Client Server Architecture in Distributed System, *GeeksforGeeks*, Last updated 23 Jul. 2025. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/three-tier-client-server-architecture-in-distributed-system/>. [Accessed: 18-Dec-2025].
- [7] P.-J. L. Michaud, "Agile software development: everything you need to know," *Nexapp*, 25 Mar. 2024. [Online]. Available: <https://www.nexapp.ca/en/blog/agile-software-development>. [Accessed: 18-Dec-2025]